

Problem Statement

Web Scraping — charities.gov.sg

Singapore Charity Portal — Advance Search Data Extraction

Field	Details
Target site	https://www.charities.gov.sg/Pages/AdvanceSearch.aspx
Data needed	Full charity list + Organisation Profile + Financial Information per charity
Total records	~2,446 registered charities (plus IPCs, de-registered, exempt, de-exempted)
Blocking issue	DOSarrest WAF rate-limiting after ~5–6 rapid requests
Current status	250 of 2,446 list records cached; detail panels not yet fetchable at scale

1. Objective

Extract a complete, structured dataset of every charity registered with the Singapore Commissioner of Charities, including:

- The searchable list (charity name, UEN, status, sector, registration date, etc.) for each of the five charity statuses offered by the Advance Search page.
- The Organisation Profile detail panel for each charity (contact details, governance board members, key officers, patrons, objectives, activities).
- The Financial Information detail panel for each charity (financial summary, receipts, expenses, balance sheet, and other information broken down by financial year).

The combined dataset should be exported to a single enriched Excel workbook per status, with properly structured sheets (not raw JSON), suitable for analysis and reporting.

2. How the Site Works (Reverse-Engineered)

The Advance Search page is a standard ASP.NET / SharePoint application. All data flows through two internal AJAX handlers, confirmed via browser DevTools network traffic capture:

2.1 List Endpoint (pagination)

```
POST /_layouts/15/CPInternet/AdvanceSearchHandler.ashx
```

- Accepts a jQuery DataTables-style form body.
- The charity-status filter lives inside the **query** field as `?advType=0&type=<CODE>` where `<CODE>` is a CRM option-set integer (e.g. 100000000 for Registered Charities).
- Pagination is controlled by `start` and `length` parameters. Each page returns up to 50 records.
- Each record includes a GUID field `CharityAccountCRMRecordID` — this is the key that links to the detail panels.

2.2 Detail Panel Endpoints (per charity)

```
GET /_layouts/15/CPInternet/SearchResultHandler.ashx?query=<Q>&type=<PANEL>
```

- `<Q>` is simply `base64(CharityAccountCRMRecordID)`. For example, the GUID `d5916f1a-7382-e711-901b-005056962860` encodes to `ZDU5MTZmMWEtNzM4Mi1lNzExLTkwMWItMDA1MDU2OTYyODYw.`
- `<PANEL>` is either **Organisation Profile** or **Financial Information**.

Confirmed example URLs captured from DevTools:

```
https://www.charities.gov.sg/_layouts/15/CPInternet/
SearchResultHandler.ashx?query=ZDU5MTZmMWEtNzM4Mi1lNz
ExLTkwMWItMDA1MDU2OTYyODYw&type=Organisation%20Profile

https://www.charities.gov.sg/_layouts/15/CPInternet/
SearchResultHandler.ashx?query=ZDU5MTZmMWEtNzM4Mi1lNz
ExLTkwMWItMDA1MDU2OTYyODYw&type=Financial%20Information
```

Both endpoints return JSON. No authentication or login is required — these are public-facing read-only APIs.

3. The Problem: WAF Rate-Limiting (DOSarrest)

The site sits behind DOSarrest, a CDN / DDoS-protection layer (similar to Cloudflare). This WAF enforces a strict per-IP rate limit that makes automated data extraction extremely difficult:

3.1 Observed behaviour

- After approximately 5–6 requests in quick succession, the WAF begins throttling.
- Throttled list requests return HTTP 200 with a valid JSON envelope but zero charity records — a silently empty page, not an error. This is indistinguishable from "no more data" without cross-checking the expected total.
- If requests continue, the WAF escalates to explicit HTTP 429 (Too Many Requests) responses.
- Once triggered, the block persists for several minutes regardless of whether new requests are sent — it is time-based, not request-count-based.

3.2 Scale of the problem

For the Registered Charities status alone:

- List pagination: ~50 requests (2,446 records ÷ 50 per page).
- Detail panels: ~4,892 requests (2 panels × 2,446 charities).
- Total: ~4,942 requests for one status.

At the rate the WAF allows (~5–6 requests before a multi-minute block), completing a full scrape in a single session is impractical. In testing, the script was blocked after fetching only 250 of 2,446 list records, and each recovery cycle (waiting out the block + retrying) added 5–10 minutes of dead time.

3.3 Screenshot: throttling in action

The screenshot below shows the scraper running with conservative settings (3 workers, 1.0s delay). After successfully fetching 250 records, every subsequent page returns 0 rows, and the script correctly identifies this as throttling and retries with exponential backoff — but the WAF never lifts the block within the retry window:

```
list: requesting rows 250-300 of 2446 ...
list: got 0 rows (expected up to 50, 2196 still remain) -- looks like throttling,
retry 1/8 in 11.9s
list: requesting rows 250-300 of 2446 ...
list: got 0 rows (expected up to 50, 2196 still remain) -- looks like throttling,
retry 2/8 in 22.0s
...
list: got 0 rows (expected up to 50, 2196 still remain) -- looks like throttling,
retry 8/8 in 153.0s
```

Even with backoff delays reaching 2.5 minutes between retries, the WAF continues blocking. The total wait across 8 retries exceeded 10 minutes with no recovery.

4. What Has Been Tried

4.1 Approach: aggressive scraping (fast, parallel)

- 6 parallel worker threads, 0.5s per-thread delay.
- Result: blocked almost immediately. Aggregate request rate (~12 req/s) far exceeds what the WAF tolerates.

4.2 Approach: conservative scraping (slow, polite)

- 2–3 workers, 1.0–1.5s delay, with a global rate limiter capping the entire process to ~24 requests/minute.
- Exponential backoff on empty pages and 429s, with cooldowns up to 5 minutes.
- Session warm-up (loading the actual search page first to collect cookies before calling XHR endpoints).
- Realistic browser headers (User-Agent, Referer, Origin, X-Requested-With).
- Result: gets further (~250 records) but still hits the wall eventually. 24 req/min is still too fast for this WAF.

4.3 Approach: extreme backoff

- Global rate dropped to 8–12 requests/minute (~5–7 seconds between requests).
- Unlimited stall retries (never gives up — just keeps waiting).
- Result: can theoretically complete, but estimated wall time for a single status is 7+ hours, and even at this rate sporadic blocks still occur, adding further dead time.

4.4 Features already built into the scraper

- Fully resumable: every API response (list pages and detail panels) is cached to disk incrementally. A crash, Ctrl+C, or WAF block never loses progress — re-running the same command picks up exactly where it left off.
- Adaptive rate limiting: the script tracks successes and failures globally and automatically slows down when pushed back, then gradually speeds up after a clean streak.
- Circuit breaker: after 3 consecutive failures across any thread, all threads pause for a configurable cooldown (default 5 minutes, escalating with each trip).
- Incremental Excel export: the export sub-command builds the .xlsx entirely from the local cache, so partial data can be exported at any time without re-scraping.

5. What Is Needed

The scraper logic, caching, resume, and Excel export are all working correctly. The sole remaining blocker is the WAF rate limit. Possible solutions include:

Option A — Rotate source IPs. Use a pool of residential or datacenter proxies so each IP stays under the per-IP rate limit while the aggregate throughput is high enough to complete in a reasonable time. A pool of 10 proxies at 5 req/min each gives 50 req/min (~1.5 hours for the full Registered list).

Option B — Longer patience window. Accept 8–12 req/min from a single IP and simply leave the script running for several hours (possibly overnight). The script already supports this — it never gives up and caches everything incrementally. This requires no infrastructure changes, just time and a stable connection.

Option C — Official data channel. Check whether the Commissioner of Charities or data.gov.sg publishes an official bulk export or API with higher rate limits. Government open-data portals sometimes carry charity datasets that would bypass the WAF entirely.

Option D — Request whitelisting. Contact the site operator (MCCY / Commissioner of Charities) and request either a higher rate-limit tier for the scraping IP, or a bulk data export. This is the cleanest solution if the data is being collected for a legitimate, non-commercial research or compliance purpose.

6. Current Deliverables

File	Description
charity_scraper.py	Main scraper script — handles list pagination, detail-panel fetching, caching, resume, and Excel export. All sub-commands (selftest, discover, scrape, export, all) are working.
list_to_excel.py	Standalone utility to build a formatted .xlsx from just the cached list JSON (no detail panels needed). Useful for exporting partial progress.
list_registered.json	Cached list data: 250 of 2,446 registered charities fetched so far.
README.md	Full technical documentation: architecture, sub-commands, CLI flags, troubleshooting.

Prepared by: **Yash Mehta**

Date: 20 August 2026